[BUG] QNGOptimizer with molecular_hamiltonian

https://github.com/PennyLaneAI/pennylane/issues/2502

ROW 81

[BUG] QNGOptimizer with molecular_hamiltonian #2502

New issue

Closed

KetpuntoG opened on Apr 27, 2022

Expected behavior

The following code was to be executed:

```
import pennylane as qml
from pennylane import qchem
from pennylane import numpy as np
import time

symbols = ["Li", "H"]
geometry = np.array([0.0, 0.0, 0.0, 0.0, 0.0, 2.969280527])

H, qubits = qchem.molecular_hamiltonian(
    symbols,
    geometry,
    active_electrons=2,
    active_orbitals=5
)

dev = qml.device("default.qubit", wires = 10)

@qml.qnode(dev)
def circuit(param):
    qml.DoubleExcitation(param[0], wires=[0,1,2,3])
    return qml.expval(H)

opt = qml.QNGOptimizer(stepsize=0.5)

param = np.array([1.], requires_grad = True)
param = opt.step(circuit, param)
print(param)
```

Actual behavior

the code does not work because this error is displayed:

```
DiagGatesUndefinedError
```

Additional information

The above error is due to the fact that for some reason the DoubleExcitation operator is not expanded in the process. Decomposing manually solves the problem.

```
import pennylane as qml
from pennylane import qchem
from pennylane import numpy as np
import time

symbols = ["Li", "H"]
geometry = np.array([0.0, 0.0, 0.0, 0.0, 0.0, 2.969280527])

H, qubits = qchem.molecular_hamiltonian(
    symbols,
    geometry,
    active_electrons=2,
    active_orbitals=5
)

dev = qml.device("default.qubit", wires = 10)
```

Assignees

KetpuntoG

Labels

bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

```
@qml.qnode(dev)
def circuit(param):
    with qml.tape.stop_recording():
        ops = qml.DoubleExcitation(param[0], wires=[0,1,2,3]).decomposition()
    for op in ops:
        qml.apply(op)
    return qml.expval(H)

opt = qml.QNGOptimizer(stepsize=0.5)

param = np.array([1.], requires_grad = True)
param = opt.step(circuit, param)
print(param)
```

However, a new error appears:

```
Operation Hamiltonian is not written in terms of a single parameter
```

This error is due to casting when obtaining the qchem Hamiltonian. It can be solved manually by adding the following line after defining the Hamiltonian:

```
H = qml.Hamiltonian([float(co) for co in H.coeffs], H.ops)
```

But this is something that should be fixed internally and not have to be done manually.

Source code

No response

Tracebacks

No response

System information

```
Name: PennyLane
Version: 0.24.0.dev0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: https://github.com/XanaduAI/pennylane
Author: None
Author-email: None
License: Apache License 2.0
Location: /Users/guille/Documents/GitHub/pennylane
Requires: numpy, scipy, networkx, retworkx, autograd, toml, appdirs, semantic-version, autoray, cachetools, pe
Required-by: PennyLane-Qchem, PennyLane-Lightning, PennyLane-SF, PennyLane-qiskit, my-research

Platform info:           macOS-10.16-x86_64-i386-64bit
Python version:          3.8.8
Numpy version:           1.20.2
Scipy version:           1.6.2
Installed devices:
- lightning.qubit (PennyLane-Lightning-0.22.1)
- strawberryfields.fock (PennyLane-SF-0.20.0.dev0)
- strawberryfields.gaussian (PennyLane-SF-0.20.0.dev0)
- strawberryfields.gbs (PennyLane-SF-0.20.0.dev0)
- strawberryfields.remote (PennyLane-SF-0.20.0.dev0)
- strawberryfields.tf (PennyLane-SF-0.20.0.dev0)
- qiskit.aer (PennyLane-qiskit-0.19.0.dev0)
- qiskit.basicaer (PennyLane-qiskit-0.19.0.dev0)
- qiskit.ibmq (PennyLane-qiskit-0.19.0.dev0)
- qiskit.ibmq.circuitrunner (PennyLane-qiskit-0.19.0.dev0)
- qiskit.ibmq.sampler (PennyLane-qiskit-0.19.0.dev0)
- default.gaussian (PennyLane-0.24.0.dev0)
- default.mixed (PennyLane-0.24.0.dev0)
- default.qubit (PennyLane-0.24.0.dev0)
- default.qubit.autograd (PennyLane-0.24.0.dev0)
- default.qubit.jax (PennyLane-0.24.0.dev0)
- default.qubit.tf (PennyLane-0.24.0.dev0)
- default.qubit.torch (PennyLane-0.24.0.dev0)
```

Existing GitHub issues

Existing GitHub issues

- [x] I have searched existing GitHub issues to make sure the issue does not already exist.

KetpuntoG added bug on Apr 27, 2022

antalszava on Apr 27, 2022 — Contributor

Thank you @KetpuntoG for the report! @soranjh any thoughts here?

> However, a new error appears:
> Operation Hamiltonian is not written in terms of a single parameter
> This error is due to casting when obtaining the qchem Hamiltonian. It can be solved manually by adding the following line after defining the Hamiltonian:
> H = qml.Hamiltonian([float(co) for co in H.coeffs], H.ops)
> But this is something that should be fixed internally and not have to be done manually.

Would this error be resolved even if `requires_grad=False` was set for the coefficients? We were discussing this with Soran and came to the conclusion, that `requires_grad=False` would have to be set by users explicitly because we would like them to have the mindset of using the differentiable HF solver.

antalszava on Apr 27, 2022 — Contributor

Having said that, some more pointers or a dedicated error message could be added to help users here.

KetpuntoG on Apr 27, 2022 — Contributor, Author

I tried that:

```
geometry = np.array([0.0, 0.0, 0.0, 0.0, 0.0, 2.969280527], requires_grad = False)
```

and the error is the same

soranjh on Apr 27, 2022 — Contributor

The bug is not particular to Hamiltonians generated with the `molecular_hamiltonian` function.

1. `QNGOptimizer` fails even if the Hamiltonian is `H = qml.Hamiltonian([1.0], [qml.Identity(wires=[0])])` when the circuit contains `qml.DoubleExcitation(param[0], wires=[0,1,2,3])`.
2. `QNGOptimizer` fails with any Hamiltonian that has differentiable coefficients, i.e., where `H.coeffs` has `requires_grad=True`.

In the Accelerating VQEs with quantum natural gradient demo a solution is provided for the second issue:

> We also need to make the Hamiltonian coefficients non-differentiable by setting requires_grad=False.
> `hamiltonian = qml.Hamiltonian(np.array(hamiltonian.coeffs, requires_grad=False), hamiltonian.ops)`

Another solution could be modifying the `QNGOptimizer` to handle differentiable Hamiltonians properly. As another solution, the Hamiltonian coefficients can be made non-differentiable by default, which is not the best option from the qchem perspective.

👍 1

KetpuntoG self-assigned this on Apr 28, 2022

KetpuntoG on Apr 28, 2022 — Contributor — Author

Okay, I'll try to make some time today to solve the problem. Thanks! 👌

👍 2

josh146 on Apr 28, 2022 · edited by josh146 — Edits — Member

@KetpuntoG I would turn this into two separate issues, since there are two separate bugs here! This way solving one of the bugs does not block closing an issue.

1. The first bug is that `qml.DoubleExcitation` returns a `Hamiltonian` when you call `DoubleExcitation.generator`. Since `Hamiltonian` does not have `diagonalizing_gates` defined, this line in the metric tensor causes an issue:

   pennylane/pennylane/transforms/metric_tensor.py
   Line 405 in b9bc9be

   ```
   405     o.diagonalizing_gates()
   ```

   Here is code to reproduce it:

   ```
   @qml.qnode(dev)
   def circuit(param):
       qml.DoubleExcitation(param[0], wires=[0,1,2,3])
       return qml.expval(qml.PauliZ(0))
   ```

   I don't know the best solution here, but it might involve forcing the metric tensor transform to decompose any operation that returns Hamiltonians as generators.

2. The other bug is related to `expval(H)`, and can be seen with this example:

   ```
   @qml.qnode(dev)
   def circuit(param):
       qml.RX(param[0], wires=0)
       return qml.expval(H)
   ```

   This is due to the `graph.iterate_parametrized_layers()` here:

   pennylane/pennylane/transforms/metric_tensor.py
   Lines 386 to 395 in b9bc9be

   ```
   386     for queue, curr_ops, param_idx, _ in graph.iterate_parametrized_layers():
   387         params_list.append(param_idx)
   388         coeffs_list.append([])
   389         obs_list.append([])
   390
   391         # for each operation in the layer, get the generator
   392         for op in curr_ops:
   393             obs, s = qml.generator(op)
   394             obs_list[-1].append(obs)
   395             coeffs_list[-1].append(s)
   ```

   In particular, because the Hamiltonian coefficients are trainable, it is being counted as a parametrized layer! I think a good solution here is to *exclude* Hamiltonians for being considered either inside `iterate_parametrized_layers()`, or within this for loop:

   pennylane/pennylane/transforms/metric_tensor.py
   Line 392 in b9bc9be

   ```
   392     for op in curr_ops:
   ```

KetpuntoG mentioned this on Apr 29, 2022

QNGOptimizer_Hamiltonian #2524

```
392         for op in curr_ops:
393             obs, s = qml.generator(op)
394             obs_list[-1].append(obs)
395             coeffs_list[-1].append(s)
```

In particular, because the Hamiltonian coefficients are trainable, it is being counted as a parametrized layer! I think a good solution here is to *exclude* Hamiltonians for being considered either inside `iterate_parametrized_layers()`, or within this for loop:

pennylane/pennylane/transforms/metric_tensor.py
Line 392 in b9bc9be

```
392     for op in curr_ops:
```

KetpuntoG mentioned this on Apr 29, 2022

QNGOptimizer_Hamiltonian #2524

antalszava on May 3, 2022 — Contributor

Hi **@KetpuntoG** just double-checking: did #2524 only partially solve this issue?

KetpuntoG on May 3, 2022 — Contributor — Author

One issue is solved and the other is solved if you use:

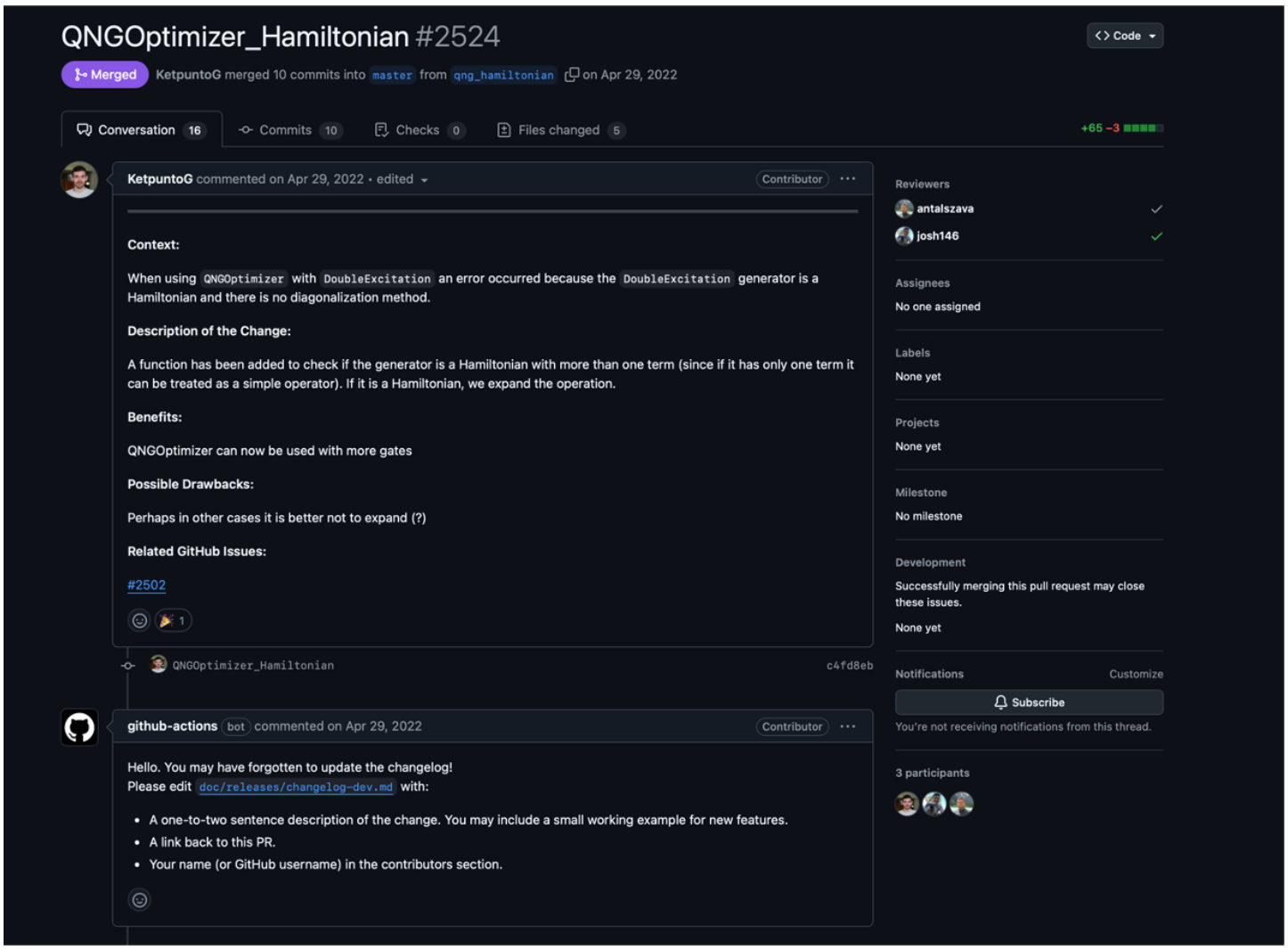
```
hamiltonian = qml.Hamiltonian(np.array(hamiltonian.coeffs, requires_grad=False), hamiltonian.ops)
```

So it is not an error as such. It's just to keep in mind that the Hamiltonian has trainable parameters by default and we should set `requires_grad = False`.

antalszava on May 10, 2022 — Contributor

Thanks, closing this.

antalszava closed this as completed on May 10, 2022


QNGOptimizer_Hamiltonian #2524
Code
Merged
KetpuntoG merged 10 commits into master from qng_hamiltonian on Apr 29, 2022
Conversation 16
Commits 10
Checks 0
Files changed 5
+65 −3
KetpuntoG commented on Apr 29, 2022 • edited
Contributor
Context:
When using QNGOptimizer with DoubleExcitation an error occurred because the DoubleExcitation generator is a Hamiltonian and there is no diagonalization method.
Description of the Change:
A function has been added to check if the generator is a Hamiltonian with more than one term (since if it has only one term it can be treated as a simple operator). If it is a Hamiltonian, we expand the operation.
Benefits:
QNGOptimizer can now be used with more gates
Possible Drawbacks:
Perhaps in other cases it is better not to expand (?)
Related GitHub Issues:
#2502
1
QNGOptimizer_Hamiltonian
c4fd8eb
github-actions bot commented on Apr 29, 2022
Contributor
Hello. You may have forgotten to update the changelog!
Please edit doc/releases/changelog-dev.md with:
A one-to-two sentence description of the change. You may include a small working example for new features.
A link back to this PR.
Your name (or GitHub username) in the contributors section.
Reviewers
antalszava
josh146
Assignees
No one assigned
Labels
None yet
Projects
None yet
Milestone
No milestone
Development
Successfully merging this pull request may close these issues.
None yet
Notifications
Customize
Subscribe
You're not receiving notifications from this thread.
3 participants